TECHNICAL DESIGN DOCUMENT

PERSONAL SONG NOTEBOOK

Full product architecture, user interface, database schema, API contracts, data lifecycle, security, backup, and implementation roadmap.

PROJECT	DECISION
Product	Private digital song notebook (not a streaming service)
Audience	One owner only - no public sign-up or multi-user accounts
Content	Notebook > song entries, rich text, notes, genre, composer, lyricist
Preferred implementation	Next.js + Tailwind + Tiptap; Laravel API; MySQL 8
Document version	1.0 - 10 October 2026

Design principle: notebook-first editing with simple navigation, reliable autosave, clean reading mode, practical print/export, and recoverable data.

1. Executive scope and assumptions

This system is a personal song notebook for a single owner. It stores written song entries and notes. It is not an audio streaming, public lyrics, licensing, social-sharing, or paid-subscription service.

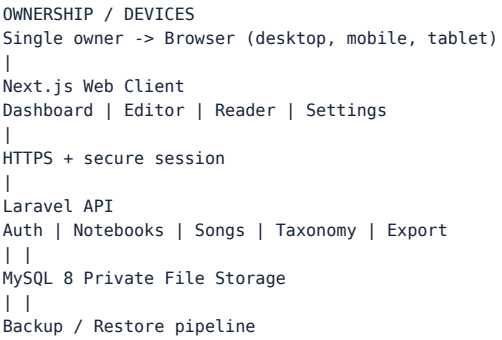
Functional scope

- Create, rename, arrange, favorite, archive/trash and restore notebooks.
- Add songs to notebooks; edit title, manually assigned song number, rich-text content and personal notes.
- Classify a song with one or more genres and credit one or more composers and lyricists.
- Find songs by title, number, content, genre, composer, lyricist and notebook.
- Move songs between notebooks; reorder entries; read, print, export and restore.
- Autosave editing with revision protection, historical snapshots and soft deletion.
- Upload optional images/documents and include them in backups.
- Responsive desktop/mobile experience; optional later offline/PWA support.

Deliberately excluded from v1

Public registration, multi-user permissions, payments, audio playback, streaming, sharing, collaborative editing, analytics dashboards and public song catalogues.

2. System architecture



Recommended stack and component responsibilities

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend	Next.js, Tailwind CSS	Notebook navigation, search, responsive UI, reading mode
Editor	Tiptap (structured JSON)	Formatting, composition, keyboard shortcuts
API	Laravel, validation, policies	CRUD, auth, search, authorization, audit
Database	MySQL 8.0+	Metadata, relationships, content, history
Storage	Laravel private storage	Attachments, exported/backup files
Search	MySQL FULLTEXT + fallback	Full-text/title/metadata filtering
Deployment	Nginx + PHP-FPM + Node	TLS, reverse proxy, queues/cron as needed

3. Screens and UX flow

ROUTE	SCREEN	CAPABILITIES
/login	Owner login	Secure credential entry, throttling, logout
/	Dashboard	Notebook cards; counts; recent entries; global search
/notebooks	All notebooks	Create, reorder, favorite and trash
/notebooks/{id}	Notebook detail	Song list, sort, filter, add, reorder, print
/songs/{id}	Reading mode	Readable entry, composer/genre credits, navigate prev/next
/songs/{id}/edit	Song editor	Rich text, title, number, credits, notes, autosave
/search	Search results	Text search + notebook/genre/composer filters
/trash	Recycle bin	Recover songs/notebooks; permanent delete confirmation
/settings	Settings	Theme, editor controls, password, backup/import/export

Editing interactions

Open a notebook -> Add Song -> fill title/number -> select genres/composers/lyricists -> edit page content and notes -> autosave -> view in reader. Genre and composer pickers permit searching existing names and creating new ones inline. The reader exposes Edit, Favorite, Print and Next/Previous.

4. Data model and relationships

```
owner (one seeded account, standalone)
notebooks 1 ------ N songs
songs 1 ------ N song_versions
songs 1 ------ N attachments
songs N ------ N genres via song_genre
songs N ------ N composers via song_composer
songs N ------ N lyricists via song_lyricist
songs N ------ N tags via song_tag
settings: singleton-style key/value table
backup_logs: backup metadata
```

A single owner removes the need for user_id on each notebook/song. Authentication still protects every endpoint. Metadata catalogues are normalized: a composer record is created once and reused for many songs. Independent many-to-many pivot tables prevent duplicate entries and allow multiple credits.

Database inventory: 15 tables

TABLE	PURPOSE
owner	Authentication; exactly one application owner
notebooks	Organized notebook collections
songs	Song metadata, editor document and search text
song_versions	Historical snapshots for recovery
attachments	Private image/file metadata
genres	Canonical genre catalogue
song_genre	Songs to genres (many to many)
composers	Canonical composer catalogue
song_composer	Songs to composers (many to many)
lyricists	Canonical lyricist catalogue
song_lyricist	Songs to lyricists (many to many)
tags	Freeform organizational labels
song_tag	Songs to tags (many to many)
settings	Application preferences
backup_logs	Backup process records

5. Detailed table definitions

5.1 owner

Seed exactly one account. Disable all public registration routes.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
id	BIGINT UNSIGNED PK	Identifier
name	VARCHAR(150) NOT NULL	Display name
email	VARCHAR(191) UNIQUE NOT NULL	Login identity
password	VARCHAR(255) NOT NULL	Argon2id/bcrypt hash only
last_login_at	TIMESTAMP NULL	Login telemetry
created_at / updated_at	TIMESTAMP NULL	Laravel timestamps

5.2 notebooks

Notebooks are collections; deleting one first moves it to Trash.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
id	BIGINT UNSIGNED PK	Notebook identifier
title	VARCHAR(200) NOT NULL	Displayed title
description	TEXT NULL	Notebook note
cover_color	VARCHAR(20) NULL	Hex/style choice
cover_image	VARCHAR(500) NULL	Private file reference
sort_order	INT NOT NULL DEFAULT 0	Manual order
is_favorite	BOOLEAN DEFAULT FALSE	Pin/favorite
created_at / updated_at	TIMESTAMP NULL	Timestamps
deleted_at	TIMESTAMP NULL	Soft deletion

5.3 songs

Main record: one song belongs to exactly one notebook. content_text is derived from content_json.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
id	BIGINT UNSIGNED PK	Song identifier
notebook_id	BIGINT UNSIGNED FK	Parent notebook
song_number	INT UNSIGNED NULL	Book-local number
title	VARCHAR(255) NOT NULL	Song heading
content_json	JSON NULL	Structured editor content
content_text	LONGTEXT NULL	Searchable plain text
notes	TEXT NULL	Private remarks
sort_order	INT DEFAULT 0	Notebook ordering
is_favorite	BOOLEAN DEFAULT FALSE	Favorite switch
revision	INT UNSIGNED DEFAULT 1	Optimistic concurrency
created_at / updated_at	TIMESTAMP NULL	Timestamps
deleted_at	TIMESTAMP NULL	Soft deletion

5.4 song_versions

Persist snapshots on meaningful change rather than every keystroke.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
id	BIGINT UNSIGNED PK	Version ID
song_id	BIGINT UNSIGNED FK	Owner song
version_number	INT UNSIGNED	Per-song sequence

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
title	VARCHAR(255)	Historical title
content_json	JSON NULL	Editor snapshot
notes	TEXT NULL	Notes snapshot
created_at	TIMESTAMP NULL	Snapshot time

5.5 attachments

Only file metadata sits in MySQL; actual bytes go to private storage.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
id	BIGINT UNSIGNED PK	Attachment
song_id	BIGINT UNSIGNED FK	Associated song
original_name	VARCHAR(255)	Original filename
file_path	VARCHAR(500)	Internal storage key
mime_type	VARCHAR(100)	MIME type
file_size	BIGINT UNSIGNED	Size in bytes
created_at	TIMESTAMP NULL	Upload time

5.6 genres / composers / lyricists / tags

Keep reusable master lists for consistent filtering.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
genres	id, name UNIQUE, description, timestamps	Multiple genres per song
composers	id, name UNIQUE, biography, timestamps	Multiple music composers
lyricists	id, name UNIQUE, biography, timestamps	Multiple text writers
tags	id, name UNIQUE, color, timestamps	Flexible labels

5.7 pivot tables

Composite primary keys make duplicate assignments impossible.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
song_genre	(song_id, genre_id) PK	Two foreign keys
song_composer	(song_id, composer_id) PK	Two foreign keys
song_lyricist	(song_id, lyricist_id) PK	Two foreign keys
song_tag	(song_id, tag_id) PK	Two foreign keys

5.8 settings / backup_logs

App-wide preferences and backup status.

COLUMN	MYSQL TYPE / CONSTRAINT	NOTES
settings	id, setting_key UNIQUE, setting_value JSON, updated_at	Preference key-value
backup_logs	id, file_path, file_size, status, created_at	Backup audit history

6. Data integrity, indexing and deletion

• Use FK constraints from songs.notebook_id to notebooks.id; from versions and attachments to songs; from each pivot to both associated tables.

• Unique song number within notebook: UNIQUE(notebook_id, song_number). MySQL allows multiple NULL values, so unnumbered drafts remain possible.

• Genre/composer/lyricist names are unique by configured collation. Normalize whitespace on input; check existing matches before creating.

• INDEX songs(notebook_id, sort_order); INDEX songs(is_favorite); FULLTEXT INDEX songs(title, content_text); INDEX notebooks(sort_order).

• Use soft deletes on notebooks and songs. Deleting a notebook must hide its songs via application scope until restored; never accidentally display orphaned trash entries.

• Do not cascade notebook deletion immediately. Permanent removal is an explicit confirmed operation with a transaction and file cleanup.

• Moving a song to a new notebook may collide with an existing number. Offer automatic next-available number, removal of number, or manually chosen replacement.

7. API structure and example payloads

METHOD / ENDPOINT	BEHAVIOR
POST /api/login; POST /api/logout	Owner session control
GET/POST /api/notebooks	List and create notebooks
GET/PATCH/DELETE /api/notebooks/{id}	Retrieve, edit, soft delete notebook
POST /api/notebooks/reorder	Persist notebook order
GET/POST /api/notebooks/{id}/songs	Notebook songs and create
GET/PATCH/DELETE /api/songs/{id}	Read, edit, soft delete song
POST /api/songs/{id}/move	Move to notebook with number handling
POST /api/songs/reorder	Reorder entries
GET /api/search	Search by text and classification
GET/POST /api/genres	List/create genres
GET/POST /api/composers	List/create composers
GET/POST /api/lyricists	List/create lyricists
GET /api/songs/{id}/versions	History
POST /api/songs/{id}/restore-version	Restore selected snapshot
GET /api/trash; POST /api/trash/restore	Trash and recovery
GET /api/export; POST /api/import	Owner export/import

Create / edit song payload (concept)

```
{
"notebook_id": 1,
"song_number": 12,
"title": "Sample Song",
"content_json": { "type": "doc", "content": [] },
"notes": "Practice notes",
"genre_ids": [2, 5],
"composer_ids": [3],
"lyricist_ids": [7],
"tag_ids": [1, 4],
"revision": 3
}
```

PATCH writes must validate referenced IDs, update song and pivot assignments in one database transaction, and use revision matching to prevent silent overwrites. API computes content_text from the sanitized editor structure rather than trusting arbitrary client-supplied plain text.

8. Security, editor and backup design

Security

• Owner login: Laravel Sanctum session/cookie authentication, HTTPS, secure HttpOnly cookie, SameSite protection and CSRF defenses.

- No public sign-up. Enforce a single owner at provisioning; never publish an owner creation endpoint.
- Validate and sanitize Tiptap JSON/HTML on server to prevent stored XSS. Limit file MIME, size and extensions; keep uploads off public executable paths.
- Throttle login and sensitive import endpoints. Secure all read and write routes with authentication middleware.
- Store secrets in environment configuration, not the repository. Back up encryption keys and credential recovery safely.

Autosave, revision and version retention

Debounce editor updates (e.g. 1-2 seconds of inactivity); show Saving/Saved/Error status. Increment revision on every accepted change. If revision mismatches, return HTTP 409 and let owner review or merge the newer copy. Keep historical versions at significant checkpoints, e.g. on explicit Save/Close or after substantial edits; prune with a configurable retention policy.

Backup and restore

Export a versioned archive containing notebook/song metadata, genre/composer/lyricist catalogues, editor JSON, attachments and a manifest/checksums. Encrypt sensitive backups where appropriate. Restore first into a staging process: verify archive format and paths, validate schema, preview conflicts, then apply atomically when practical. Test recovery routinely. A database-only export does not preserve attachments.

Offline / PWA - later phase

If implemented, cache the app shell and store pending edits in IndexedDB with a durable outbox. Sync via idempotent mutation IDs and revision checks; display offline status and resolve conflicts before discarding local edits. This is not automatic just by installing a PWA.

9. Implementation stages and acceptance criteria

PHASE	FEATURE SET	ACCEPTANCE CHECK
1: Foundation	Login, schema, notebook CRUD	Owner-only access, valid FK rules, trash
2: Notebook editing	Song CRUD, rich editor, autosave	Content survives refresh and bad connection
3: Metadata/search	Genres, composers, lyricists, tags, search	Filtering by combinations produces correct songs
4: Quality	History, favorite, reorder, move	Conflict/restore/reorder verified
5: Export/recovery	Print, PDF, backup + restore	Clean print; test full archive restore
6: Optional	Offline PWA, attach images	Offline edits safely reconcile

Critical test cases

- Two songs can use the same number in different notebooks, but not in one notebook.
- A song can have multiple composers and multiple genres; duplicate pivot entries are rejected.
- Moving a song to a notebook with the same number gives an explicit resolution path.
- Deleted notebook content does not appear in normal search, but can be restored.
- Concurrent stale edit generates conflict instead of erasing more recent content.

- Import rejects unsafe attachment paths and incorrect backup schema version.

10. SQL reference - relational master tables

The following is a compact SQL reference for the metadata and pivot layer. Core owner/notebooks/songs definitions are listed in Section 5 and should be implemented as Laravel migrations with the documented data types, indexes and constraints.

```
CREATE TABLE genres (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL UNIQUE,
description TEXT NULL,
created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL
);
CREATE TABLE composers (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(200) NOT NULL UNIQUE,
biography TEXT NULL,
created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL
);
CREATE TABLE lyricists (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(200) NOT NULL UNIQUE,
biography TEXT NULL,
created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL
);
CREATE TABLE tags (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL UNIQUE,
color VARCHAR(20) NULL,
created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL
);
CREATE TABLE song_genre (
song_id BIGINT UNSIGNED NOT NULL,
genre_id BIGINT UNSIGNED NOT NULL,
PRIMARY KEY(song_id, genre_id),
FOREIGN KEY(song_id) REFERENCES songs(id) ON DELETE CASCADE,
FOREIGN KEY(genre_id) REFERENCES genres(id) ON DELETE CASCADE
);
CREATE TABLE song_composer (
song_id BIGINT UNSIGNED NOT NULL,
composer_id BIGINT UNSIGNED NOT NULL,
PRIMARY KEY(song_id, composer_id),
FOREIGN KEY(song_id) REFERENCES songs(id) ON DELETE CASCADE,
FOREIGN KEY(composer_id) REFERENCES composers(id) ON DELETE CASCADE
);
CREATE TABLE song_lyricist (
song_id BIGINT UNSIGNED NOT NULL,
lyricist_id BIGINT UNSIGNED NOT NULL,
PRIMARY KEY(song_id, lyricist_id),
FOREIGN KEY(song_id) REFERENCES songs(id) ON DELETE CASCADE,
FOREIGN KEY(lyricist_id) REFERENCES lyricists(id) ON DELETE CASCADE
);
CREATE TABLE song_tag (
song_id BIGINT UNSIGNED NOT NULL,
tag_id BIGINT UNSIGNED NOT NULL,
PRIMARY KEY(song_id, tag_id),
FOREIGN KEY(song_id) REFERENCES songs(id) ON DELETE CASCADE,
FOREIGN KEY(tag_id) REFERENCES tags(id) ON DELETE CASCADE
);
```

11. Practical implementation notes

Recommended v1 model: Each song is one editable entry within a notebook; a dedicated pages table is not needed. If the product later requires arbitrary pages, cover pages, dividers or several songs on one sheet, introduce notebook_pages with page_type and position rather than overloading songs.

Composer versus lyricist: Composer means melody/music creator; lyricist means text writer. A single person may appear in both catalogues, intentionally, unless future contributor normalization is requested.

Document completeness: This PDF provides the full logical architecture and database design, example SQL for classification tables, validation/constraint rules and implementation plan. Developers should generate complete application migrations, API validators, tests and deploy configuration during implementation.